

Práctica S1: Desmantelando el Monolito de Control - Jorge Parra ITIT 13104

Tarea 1. El mapa del colapso

Pregunta: ¿Qué pasa con la libertad de información si el cargador de videos se satura por un ataque de denegación de servicio (DDoS)? ¿Aun así se pueden ver los videos que ya existen?

En un monolito, si una parte se cae o se satura, es muy probable que afecte a todo el sistema. Entonces, aunque los videos ya estén guardados, puede que **no se puedan reproducir** porque el streaming también termina fallando.

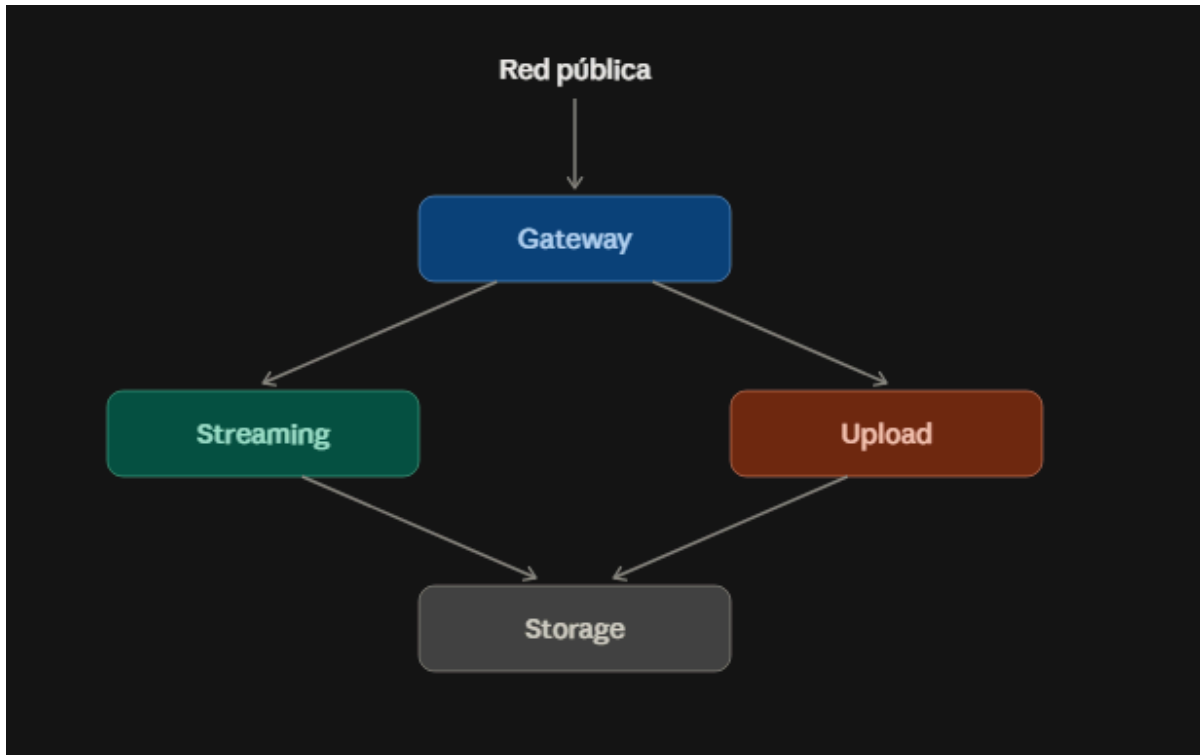
En microservicios, como cada parte corre por separado, un problema en Upload no necesariamente tumba Streaming.

Tarea 2. El plano de la redundancia

La propuesta es dividir el sistema en **4 microservicios** (cada uno con una responsabilidad clara):

¿Qué es cada componente?

- **Red pública:** Internet / usuarios externos que acceden al sistema.
- **Gateway:** “puerta de entrada” que recibe las peticiones y las manda al servicio correcto (también puede hacer cosas como autenticación o rate limit).
- **Streaming:** servicio que entrega/reproduce los videos a los usuarios.
- **Upload:** servicio que recibe y procesa la subida de nuevos videos.
- **Storage:** almacenamiento donde se guardan los archivos de video (y/o sus datos), usado por Upload y Streaming.



Conexiones

- Red Pública → Gateway
- Gateway → Streaming
- Gateway → Upload
- Streaming → Storage
- Upload → Storage

Restricciones / reglas

- **Streaming y Upload no se hablan directo** entre ellos.
- **Gateway** es lo único expuesto a la red pública.
- Storage se mantiene aislado y solo se usa por las conexiones definidas.
- (En una versión "real") lo ideal es que **cada microservicio tenga su propia base de datos** para evitar dependencia y acoplamiento.

Ventaja principal: si **Upload falla o se satura**, **Streaming puede seguir funcionando** y la gente puede ver videos que ya estaban guardados.

Tarea 3. Simulación de interceptación

1) Misión de extracción (GET en modo verbose)

Objetivo: consultar el video `sunstone-01` con salida detallada.

```
curl -v "http://192.168.1.50:8080/video?id=sunstone-01"
```

El `-v` muestra más info de la comunicación HTTP (conexión, cabeceras y la respuesta).

¿Qué significa un HTTP 500?

HTTP 500 (Internal Server Error) = el servidor tuvo un error interno y no pudo completar la solicitud.

En un monolito, esto se vuelve grave porque si un componente se satura (por ejemplo Upload), puede arrastrar al resto y terminar afectando también el Streaming.

En microservicios, el daño se "encierra" más: si Upload se rompe, Streaming puede seguir respondiendo.

2) Misión de inyección de metadatos (POST)

Objetivo: registrar un video enviando metadatos con `POST`.

```
curl -X POST "http://192.168.1.50:3000/api/videos" -H "Content-Type: application/json" -d '{"title":"Mensaje de El","duration":120}'
```

¿Qué hace cada parte?

- `-X POST`: manda una solicitud tipo POST.
- `-H "Content-Type: application/json"`: dice que el cuerpo va en JSON.
- `-d ...`: el cuerpo (los datos) que se mandan.

Resultado esperado: el servidor recibe los datos y los procesa en este endpoint:

```
http://192.168.1.50:3000/api/videos
```

Con esto se ve el uso básico de **GET** y **POST** con `curl`.